

Laporan Tugas Besar 1 IF3170 Pembelajaran Mesin

Feedforward Neural Network



Disusun oleh:

Ahmad Ibrahim	13523089
Michael Alexander Angkawijaya	13523102
Aria Judhistira	13523112

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132**

2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
I. DESKRIPSI PERSOALAN.....	3
II. PEMBAHASAN.....	4
2.1 Penjelasan Implementasi.....	4
2.1.1 Kelas ActivationFunction.....	4
2.1.2 Kelas LossFunction.....	4
2.1.3 Kelas Layer.....	5
2.1.4 Kelas Initializer.....	6
2.1.5 Kelas FFNN.....	7
2.1.6 Kelas ModelConfig.....	8
2.1.7 Forward Propagation.....	9
2.1.8 Backward Propagation.....	10
2.1.9 Kelas RMSNorm.....	12
2.1.10 Kelas Tensor.....	12
2.2 Hasil Pengujian.....	14
2.2.1 Pengaruh Depth dan Width.....	15
2.2.2 Pengaruh Fungsi Aktivasi.....	16
2.2.3 Pengaruh Learning Rate.....	18
2.2.4 Pengaruh Inisialisasi Bobot.....	20
2.2.5 Pengaruh Regularisasi.....	20
2.2.6 Pengaruh Normalisasi RMSNorm.....	22
2.2.7 Perbandingan dengan Library sklearn.....	22
III. KESIMPULAN DAN SARAN.....	23
IV. PEMBAGIAN TUGAS.....	24
V. REFERENSI.....	25
VI. LAMPIRAN.....	26

I. DESKRIPSI PERSOALAN

Tujuan tugas besar ini adalah untuk mengimplementasikan sebuah *Feedforward Neural Network* (FFNN) *from scratch*. Implementasi dari FFNN mengikuti ketentuan-ketentuan sebagai berikut.

- FFNN yang diimplementasikan dapat menerima sejumlah neuron dari setiap *layer*, termasuk *input* dan *output layer*.
- FFNN yang diimplementasikan dapat menerima fungsi aktivasi dari setiap *layer*. Fungsi aktivasi meliputi Linear, ReLU, Sigmoid, Tanh, dan Softmax.
- FFNN yang diimplementasikan dapat menerima fungsi *loss* berupa MSE, Binary Cross-Entropy, atau Categorical Cross-Entropy.
- FFNN memiliki mekanisme inisialisasi bobot dan bias untuk setiap neuron. Inisialisasi bobot dapat berupa *zero initialization*, *random* dengan distribusi uniform, atau *random* dengan distribusi normal.
- Instansi dari model FFNN harus dapat menyimpan bobot dan gradien bobot dari setiap neuron, termasuk bias. Selain itu, instansi model harus memiliki metode untuk menampilkan distribusi bobot dan distribusi gradien bobot dari setiap *layer*.
- Terdapat cara untuk menyimpan dan memuat ulang instansi dari suatu model.
- FFNN memiliki implementasi *forward propagation* yang dapat menerima *input* berupa *batch*.
- FFNN memiliki implementasi *backward propagation* untuk menghitung perubahan gradien yang dapat menangani *input* berupa *batch*.
- FFNN memiliki implementasi metode regularisasi L1 dan L2.
- FFNN dapat melakukan pembaruan bobot dengan menggunakan *gradient descent*.
- FFNN dapat menerima parameter berupa Batch Size, Learning Rate, Jumlah Epoch, dan Verbosity.

Setelah mengimplementasikan model FFNN sepenuhnya, dilakukan pengujian pada sebuah *dataset*. Pengujian dilakukan guna menganalisis pengaruh *hyperparameter* sebagai berikut.

- Pengaruh *depth* dan *width* dari FFNN.
- Pengaruh fungsi aktivasi pada *hidden layer*.
- Pengaruh dari *learning rate*.

Selain pengaruh *hyperparameter*, adapun analisis yang dilakukan untuk mengetahui pengaruh dari regularisasi L1 dan L2 terhadap kinerja model. Terakhir, dilakukan uji perbandingan dengan model `MLPClassifier` dari *library* `sklearn`.

II. PEMBAHASAN

2.1 Penjelasan Implementasi

Berikut adalah penjelasan implementasi model FFNN yang meliputi penjelasan kelas yang telah dibuat, penjelasan mengenai *forward propagation*, dan penjelasan mengenai *backward propagation* serta *weight update*.

2.1.1 Kelas ActivationFunction

Kelas ActivationFunction merupakan sebuah *abstract class* yang berfungsi sebagai implementasi fungsi-fungsi aktivasi. Terdapat lima kelas turunan dari kelas ini, yakni kelas Linear, Sigmoid, ReLU, Tanh, dan Softmax. Masing-masing kelas turunan mengimplementasikan dua metode, yakni metode *activate()* dan *derivative()*.

Metode	Penjelasan
<code>activate(z)</code>	Mengembalikan hasil fungsi aktivasi pada parameter <i>net input</i> untuk <i>forward pass</i> .
<code>derivative(z)</code>	Mengembalikan gradien berdasarkan turunan dari fungsi aktivasi pada parameter <i>net input</i> untuk <i>backward pass</i> .

2.1.2 Kelas LossFunction

Kelas LossFunction merupakan sebuah *abstract base class* yang digunakan untuk mengimplementasikan fungsi-fungsi *loss* pada kelas-kelas turunannya. Terdapat tiga kelas yang mewarisi kelas ini, yakni kelas MSE, BinaryCrossEntropy, dan CategoricalCrossEntropy. Setiap kelas turunan mengimplementasikan dua metode, yakni *loss()* dan *derivative()*.

Metode	Penjelasan
<code>loss(y_true, y_pred)</code>	Mengembalikan <i>error</i> dari prediksi model dibandingkan dengan nilai aslinya.
<code>derivative(y_true, y_pred)</code>	Mengembalikan gradien dari <i>error</i> untuk <i>backward pass</i> .

2.1.3 Kelas Layer

Kelas Layer merupakan implementasi dari sebuah *layer* atau lapisan neuron pada model FFNN. Berikut adalah penjelasan atribut dan metode pada kelas ini.

Atribut	Penjelasan
input_dim	Dimensi atau ukuran <i>input</i> .
output_dim	Dimensi atau ukuran <i>output</i> dan jumlah neuron pada lapisan ini.
activation_fn	Fungsi aktivasi yang digunakan dalam bentuk sebuah objek <code>ActivationFunction</code> .
W	Matriks yang berisi bobot-bobot.
b	<i>List</i> yang berisi nilai bias dengan ukuran sebanyak neuron pada lapisan.
input	Matriks yang berisi nilai-nilai <i>input</i> lapisan neuron.
z	Matriks yang berisi <i>net input</i> setiap neuron.
a	Matriks yang berisi nilai <i>output</i> setiap neuron berdasarkan fungsi aktivasi.
dW	Matriks yang berisi gradien bobot setiap neuron.
db	<i>List</i> yang berisi gradien bias setiap neuron.
Metode	Penjelasan
<code>__init__(self, input_dim, output_dim, activation_fn, weights)</code>	Menginisialisasi sebuah objek Layer dan atribut-atributnya.
<code>forward(self, input_data)</code>	Melakukan <i>forward pass</i> dan mengembalikan nilai <i>output</i> setiap neuron.
<code>backward(self, d_a, use_combined)</code>	Melakukan <i>backward pass</i> dengan menghitung gradien dari total <i>error</i> terhadap nilai <i>net input</i> menggunakan turunan fungsi aktivasi, lalu menghitung gradien bobot dan bias. Mengembalikan nilai gradien yang didapatkan agar menjadi <i>input</i> (<i>d_a</i>) untuk lapisan sebelumnya.

2.1.4 Kelas Initializer

Kelas Initializer merupakan sebuah *abstract base class* yang digunakan untuk menginisialisasi bobot yang akan digunakan pada model. Terdapat tujuh kelas yang mewarisi kelas ini, yakni ZerosInitializer, RandomUniformInitializer, RandomNormalInitializer, XavierUniformInitializer, XavierRandomInitializer, HeUniformInitializer, dan HeRandomInitializer. Setiap kelas turunan mengimplementasikan metode *initialize()*.

Atribut	Penjelasan
seed	<i>Seed</i> yang digunakan untuk pembangkit nilai acak. Disimpan agar dapat mereproduksi nilai-nilai bobot yang acak.
RandomUniformInitializer	
low	Batas bawah dari bobot acak.
high	Batas atas dari bobot acak.
RandomNormalInitializer	
mean	Rata-rata pada distribusi normal.
std	Standar deviasi pada distribusi normal.
XavierNormalInitializer	
mean	Rata-rata dengan nilai standar 0.
HeNormalInitializer	
mean	Rata-rata dengan nilai standar 0.
Metode	Penjelasan
<code>__init__(self, seed)</code>	Menginisialisasi sebuah objek Initializer dengan atribut <i>seed</i> .
<code>initialize(self, shape)</code>	Mengembalikan sebuah <i>list</i> berisi nilai bobot dengan ukuran parameter <i>shape</i> .
RandomUniformInitializer	
<code>__init__(self, low, high, seed)</code>	Menginisialisasi sebuah objek RandomUniformInitializer dengan atribut <i>low</i> , <i>high</i> , dan <i>seed</i> .

RandomNormalInitializer	
<code>__init__(self, mean, variance, seed)</code>	Menginisialisasi sebuah objek RandomNormalInitializer dengan atribut <i>mean</i> , <i>variance</i> , dan <i>seed</i> .
XavierUniformInitializer	
<code>__init__(self, seed)</code>	Menginisialisasi sebuah objek XavierUniformInitializer dengan atribut <i>seed</i> .
XavierNormalInitializer	
<code>__init__(self, mean, seed)</code>	Menginisialisasi sebuah objek XavierUniformInitializer dengan atribut <i>mean</i> dan <i>seed</i> .
HeUniformInitializer	
<code>__init__(self, seed)</code>	Menginisialisasi sebuah objek HeUniformInitializer dengan atribut <i>seed</i> .
HeNormalInitializer	
<code>__init__(self, mean, seed)</code>	Menginisialisasi sebuah objek HeUniformInitializer dengan atribut <i>mean</i> dan <i>seed</i> .

2.1.5 Kelas FFNN

Kelas FFNN merupakan implementasi dari model FFNN yang mampu melakukan pelatihan model dan prediksi. Berikut adalah deskripsi dari atribut dan metode yang dimiliki kelas ini.

Atribut	Penjelasan
<code>config</code>	Objek ModelConfig yang menyimpan konfigurasi model, seperti <i>learning rate</i> dan regularisasi.
<code>initializer</code>	Objek Initializer yang digunakan untuk menginisialisasi bobot.
<code>layers</code>	Lapisan-lapisan neuron pada model FFNN.
<code>history</code>	Data riwayat berupa <i>error</i> model saat latihan dan <i>error</i> model saat validasi.
Metode	Penjelasan

<code>__init__(self, config, initializer)</code>	Menginisialisasi sebuah objek FFNN dan atribut-atributnya.
<code>_initialize_layers(self)</code>	Menginisialisasi lapisan-lapisan neuron dalam bentuk <i>list</i> yang berisi objek-objek Layer.
<code>forward(self, x)</code>	Melakukan <i>forward propagation</i> menggunakan data <i>x</i> dan mengembalikan hasil prediksi.
<code>backward(self, y_true, y_pred)</code>	Melakukan <i>backward propagation</i> dengan menghitung gradien prediksi dengan <i>label</i> asli, kemudian menghitung gradien bobot dan bias pada setiap lapisan neuronnya.
<code>_update_weights(self)</code>	Memperbarui bobot dan bias pada setiap lapisan dengan menggunakan gradien bobot, gradien bias, <i>learning rate</i> , dan parameter regularisasi.
<code>fit(self, X_train, y_train, X_val, y_val)</code>	Melakukan proses pelatihan model yang meliputi <i>forward propagation</i> dan <i>backward propagation</i> dengan pembaruan nilai bobot. Mengembalikan atribut <i>history</i> .

2.1.6 Kelas ModelConfig

Kelas ModelConfig merupakan sebuah kelas yang digunakan untuk menyimpan konfigurasi model FFNN. Konfigurasi ini disimpan dalam atribut-atribut kelas dengan deskripsi sebagai berikut.

Atribut	Penjelasan
<code>layers_dims</code>	<i>List</i> yang berisi jumlah neuron pada setiap lapisan.
<code>activation_fn</code>	<i>List</i> yang berisi jenis fungsi aktivasi setiap lapisan neuron dalam bentuk objek ActivationFunction.
<code>loss_fn</code>	Objek LossFunction yang menyatakan jenis fungsi loss yang digunakan.
<code>learning_rate</code>	Nilai dari <i>learning rate</i> model.
<code>epochs</code>	Jumlah <i>epoch</i> atau iterasi.
<code>batch_size</code>	Jumlah sampel data yang diproses pada saat bersamaan.
<code>regularization</code>	Jenis regularisasi yang digunakan (L1 atau L2).

reg_lambda	Koefisien penalti regularisasi.
verbose	Tingkat detail informasi yang ingin ditampilkan sepanjang proses pelatihan model (0 atau 1).
random_state	<i>Seed</i> untuk pembangkit nilai acak untuk memastikan <i>reproducibility</i> .
rmsnorm	<i>Flag</i> untuk menggunakan normalisasi RMSNorm.

2.1.7 Forward Propagation

Dalam sebuah model FFNN, *forward propagation* merupakan proses model untuk melakukan prediksi awal terhadap suatu data. Pada akhir proses ini, prediksi awal tersebut akan dibandingkan dengan label aktual data yang akan menghasilkan *error*. Nilai *error* ini pun akan dimanfaatkan oleh proses kedua dalam pembelajaran model, yakni *backward propagation*, untuk meningkatkan performa model.

Forward propagation merupakan proses inferensi pada model FFNN. Secara konsep, *forward propagation* mulai dari lapisan *input* yang digunakan untuk menghitung *net input* pada *hidden layer* berikutnya yang berisi neuron-neuron. Hasil dari perhitungan fungsi aktivasi dengan nilai *net input* tersebut sebagai masukan fungsi menjadi *output* neuron tersebut. Nilai keluaran bersama dengan nilai bobot pada keluaran tersebut menjadi *input* untuk neuron-neuron pada *hidden layer* berikutnya. Proses ini terus berlangsung hingga mencapai *output layer* yang akan menghasilkan nilai prediksi.

Berikut adalah alur proses *forward propagation* yang telah diimplementasikan pada program.

class FFNN
<pre>def forward(self, x: np.ndarray, return_tensor: bool = False) -> Tensor np.ndarray: x_t = Tensor(x, requires_grad=False) for layer in self.layers: x_t = layer.forward(x_t) if return_tensor: return x_t return x_t.numpy()</pre>
class Layer
<pre>def forward(self, input_data: Tensor) -> Tensor:</pre>

```

self.input = input_data
self.z = self.input @ self.W + self.b

if self.rmsnorm is not None:
    self.z = self.rmsnorm.forward(self.z)

self.a = self.activation_fn.activate(self.z)
return self.a

```

Proses *forward propagation* dimulai dari metode `FFNN.forward()` pada kelas `FFNN`. Objek *layer* pada model memanggil metode `Layer.forward()` sendiri. Pada metode tersebut, nilai *net input* setiap neuron dihitung menggunakan bobot-bobot dan bias. Kemudian, nilai *net input* yang didapatkan akan dimasukkan dalam fungsi aktivasi. Hasilnya adalah sebuah objek `Tensor` yang digunakan sebagai nilai *input* untuk lapisan berikutnya. Proses ini dilakukan secara terus-menerus hingga mencapai lapisan terakhir, yakni *output layer*, di mana `FFNN.forward()` mengembalikan *output* dalam bentuk objek `Tensor` atau larik `NumPy`.

2.1.8 Backward Propagation

Dalam model `FFNN`, *backward propagation* merupakan proses pembelajaran model yang memanfaatkan nilai prediksi dari *forward propagation* dan membandingkannya dengan nilai label sesungguhnya. Hasil perbandingan menggunakan sebuah *loss function* menghasilkan nilai *error*. Dengan memanfaatkan prinsip *gradient descent*, nilai kesalahan ini diantarkan ke lapisan sebelumnya dalam bentuk turunan parsial terhadap *output*. Setelah itu, dengan menggunakan aturan rantai, didapatkan gradien *error* terhadap bobot-bobot lapisan tersebut dan terhadap *input*. Gradien terhadap bobot kemudian digunakan untuk memperbarui bobot, sedangkan gradien terhadap *input* diantarkan ke lapisan sebelumnya untuk digunakan pada aturan rantai. Proses ini berlanjut hingga setiap lapisan telah memperbarui bobotnya.

Berikut adalah alur proses *backward propagation* yang telah diimplementasikan pada program.

```

class FFNN

def backward(self, y_true: Tensor, y_pred: Tensor) -> None:
    loss_t = self.config.loss_fn.loss(y_true, y_pred)
    loss_t.backward()
    grad = None
    for layer in reversed(self.layers):
        grad = layer.backward(grad)

def _update_weights(self) -> None:

```

```

lr = self.config.learning_rate
reg = self.config.regularization
lam = self.config.reg_lambda

for layer in self.layers:
    if reg == 'l1':
        reg_grad = lam * np.sign(layer.W.data)
    elif reg == 'l2':
        reg_grad = lam * layer.W.data
    else:
        reg_grad = 0.0

    w_grad = layer.dW if layer.dW is not None else 0.0
    b_grad = layer.db if layer.db is not None else 0.0

    layer.W.data -= lr * (w_grad + reg_grad)
    layer.b.data -= lr * b_grad

    if layer.rmsnorm is not None and layer.rmsnorm.gamma.grad is
not None:
        layer.rmsnorm.gamma.data -= lr * layer.rmsnorm.gamma.grad

```

class Layer

```

def backward(self) -> np.ndarray | None:
    self.dW = self.W.grad
    self.db = self.b.grad

    if self.input is None:
        raise ValueError("Layer input is not available")
    return self.input.grad

```

class Tensor

```

def backward(self, grad: np.ndarray | None = None) -> None:
    if not self.requires_grad:
        return
    if grad is None:
        grad = np.ones_like(self.data)
    self.grad = grad

    topo = []
    visited = set()

    def build(v):
        if v not in visited:
            visited.add(v)

```

```

        for child in v._prev:
            build(child)
        topo.append(v)

    build(self)
    for node in reversed(topo):
        node._backward()

```

Proses *backward propagation* dimulai pada pemanggilan metode `FFNN.backward()`. Berbeda dengan *backward propagation* konvensional, implementasi program dengan *automatic differentiation* menggunakan Tensor berarti setiap operasi di setiap lapisan neuron pada *forward propagation* sudah mendaftarkan operasi *backward*-nya pada graf komputasional. Hal ini memungkinkan Tensor untuk langsung menjalankan metode `_backward()` pada setiap operasi yang telah dilakukan ketika `Tensor.backward()` dipanggil. Setelah itu, setiap lapisan neuron memanggil metode `Layer.backward()` untuk memperbarui nilai gradien bobot dan bias. Setelah proses ini selesai, dipanggil metode `FFNN._update_weights()` yang memperbarui nilai bobot dan bias pada setiap *layer* dengan memanfaatkan nilai gradien bobot dan bias yang sebelumnya telah dihitung.

2.1.9 Kelas RMSNorm

Kelas `RMSNorm` merupakan kelas yang digunakan untuk membuat lapisan normalisasi `RMSNorm`.

Atribut	Penjelasan
dimension	Ukuran <i>input</i> .
epsilon	Nilai <i>epsilon</i> yang digunakan pada normalisasi.
gamma	Tensor yang berisi nilai 1.
Metode	Penjelasan
forward(z)	Memanggil metode <i>rmsnorm</i> pada Tensor dan mengalikannya dengan <i>gamma</i> .
zero_grad(z)	Memanggil metode <i>zero_grad</i> pada Tensor.

2.1.10 Kelas Tensor

Kelas `Tensor` merupakan sebuah kelas yang digunakan untuk merepresentasikan data numerik yang mendukung *automatic differentiation*.

Objek Tensor menyimpan nilai, informasi gradient, serta relasi antar operasi sehingga gradient dapat dihitung secara otomatis saat proses backward dilakukan.

Atribut	Penjelasan
data	Array NumPy yang menyimpan nilai numerik dari tensor.
grad	Gradient dari tensor terhadap fungsi objektif. Nilainya None sebelum proses backward dijalankan.
requires_grad	Penanda apakah tensor perlu menyimpan dan menghitung gradient.
_backward	Fungsi lokal yang menyimpan aturan backward untuk operasi yang menghasilkan tensor tersebut.
_prev	Himpunan tensor parent yang menjadi input pembentuk tensor saat ini dalam computational graph.
_op	String penanda operasi yang menghasilkan tensor, misalnya '+', '*', '@', 'relu', dan sebagainya.
Metode	Penjelasan
<code>__init__(data, requires_grad=False, _children=(), _op="")</code>	Membentuk objek Tensor baru dari data numerik dan informasi graph.
<code>__add__(other)</code>	Menjalankan operasi penjumlahan antar tensor dan membentuk node baru pada graph.
<code>__sub__(other)</code>	Menjalankan operasi pengurangan antar tensor.
<code>__mul__(other)</code>	Menjalankan operasi perkalian elemen per elemen antar tensor.
<code>__truediv__(other)</code>	Menjalankan operasi pembagian elemen per elemen antar tensor.
<code>__matmul__(other)</code>	Menjalankan operasi perkalian matriks antar tensor.
<code>__pow__(power)</code>	Menjalankan operasi perpangkatan tensor.
<code>sum(axis=None, keepdims=False)</code>	Menjumlahkan elemen tensor pada sumbu tertentu.

<code>mean(axis=None, keepdims=False)</code>	Menghitung rata-rata elemen tensor pada sumbu tertentu.
<code>T()</code>	Menghasilkan transpose dari tensor.
<code>exp()</code>	Menghasilkan eksponensial dari tensor.
<code>log()</code>	Menghasilkan logaritma natural dari tensor.
<code>sigmoid()</code>	Menerapkan fungsi aktivasi sigmoid pada tensor.
<code>tanh()</code>	Menerapkan fungsi aktivasi hyperbolic tangent pada tensor.
<code>relu()</code>	Menerapkan fungsi aktivasi ReLU pada tensor.
<code>softmax()</code>	Menerapkan fungsi aktivasi softmax pada tensor.
<code>backward(grad=None)</code>	Menjalankan backpropagation pada computational graph untuk menghitung gradient semua tensor yang terhubung.
<code>numpy()</code>	Mengembalikan isi data dalam bentuk array NumPy.
<code>zero_grad()</code>	Mengosongkan gradient tensor dengan mengatur grad menjadi None.
<code>rmsnorm(eps)</code>	Menerapkan normalisasi RMSNorm pada tensor.

2.2 Hasil Pengujian

Pengujian dilakukan pada model FFNN untuk membandingkan pengaruh depth-width, fungsi aktivasi, learning rate, dan regularisasi terhadap performa klasifikasi. Evaluasi dilakukan dengan melihat metrik prediksi akhir (*accuracy*, *precision*, *recall*, F1), kurva training-validation loss per epoch, serta distribusi bobot dan gradien bobot pada setiap hidden layer.

Pengujian dilakukan dengan menggunakan parameter-parameter sebagai berikut:

Fungsi aktivasi layer output = Sigmoid
 Fungsi Loss = BinaryCrossEntropy
 Weight initializer = 'RandomNormalInitializer, dengan mean = 0.0 dan variance = 0.01
 Dimensi hidden layer = [64, 64, 64]
 Fungsi aktivasi seluruh hidden layer = ReLU
 Learning rate = 0.001
 Regularization = 'none'

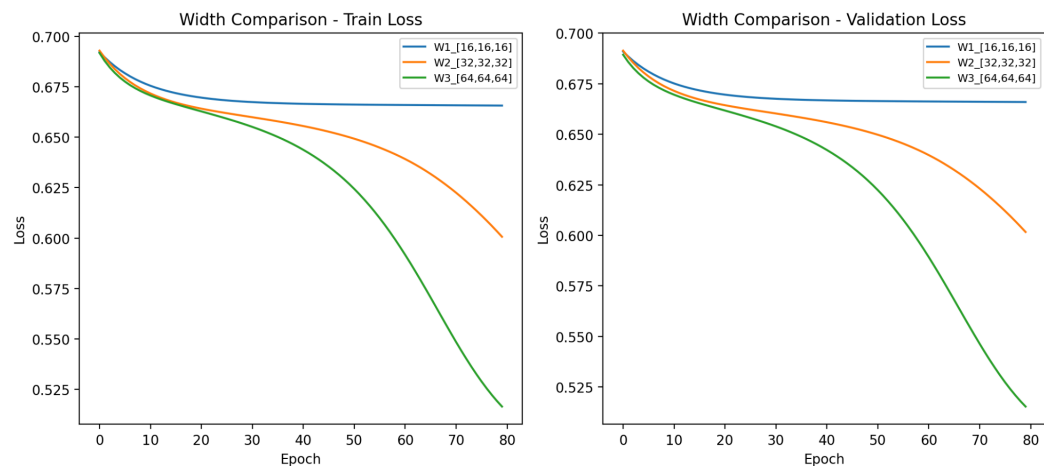
Epochs = 80
Batch size = 32
Random state = 42

Selain parameter yang diuji, akan menggunakan parameter-parameter tersebut.

2.2.1 Pengaruh Depth dan Width

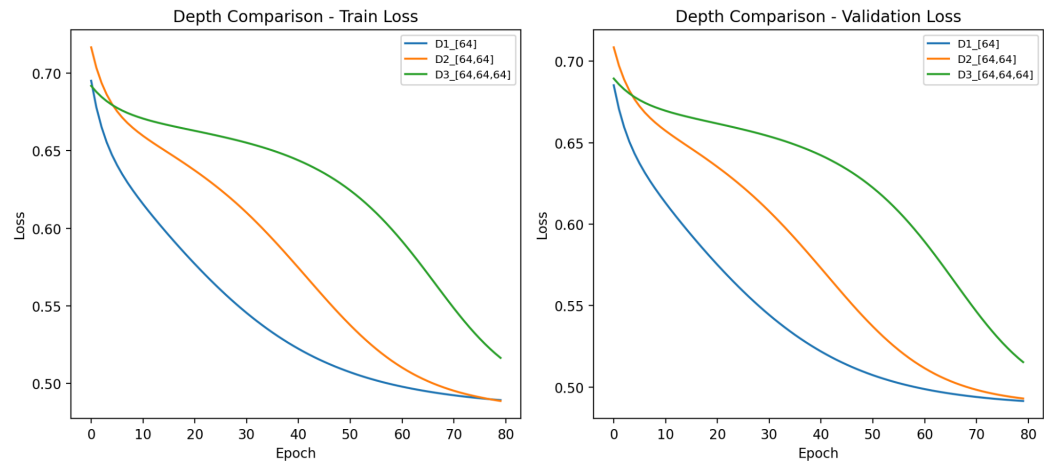
Dilakukan pengujian untuk tiga variasi kombinasi width dengan depth tetap yaitu 3.

Width	Accuracy	Precision	Recall	F1
64	0.7475	0.7545	0.8740	0.8099
32	0.647	0.6370	0.9910	0.7755
16	0.6155	0.6155	1.0	0.7619



Selanjutnya, dilakukan pengujian untuk tiga variasi kombinasi depth (jumlah *hidden layer*) dengan width tetap yaitu 64.

Depth	Accuracy	Precision	Recall	F1
1	0.7565	0.7715	0.8586	0.8127
2	0.7525	0.7709	0.8505	0.8088
3	0.7475	0.7545	0.8740	0.8099



Dari hasil pengujian, didapatkan bahwa semakin besar *width* atau jumlah neuron setiap *hidden layer*, semakin baik performa model. Dengan menambahkan lebih banyak neuron, akurasi model meningkat signifikan.

Sebaliknya, semakin kecil *depth* atau jumlah *hidden layer*, semakin baik performa model. Hidden layer yang semakin banyak justru memberikan noise tambahan pada model. Namun, perubahan *depth* terhadap akurasi model kurang signifikan dibandingkan uji *width*.

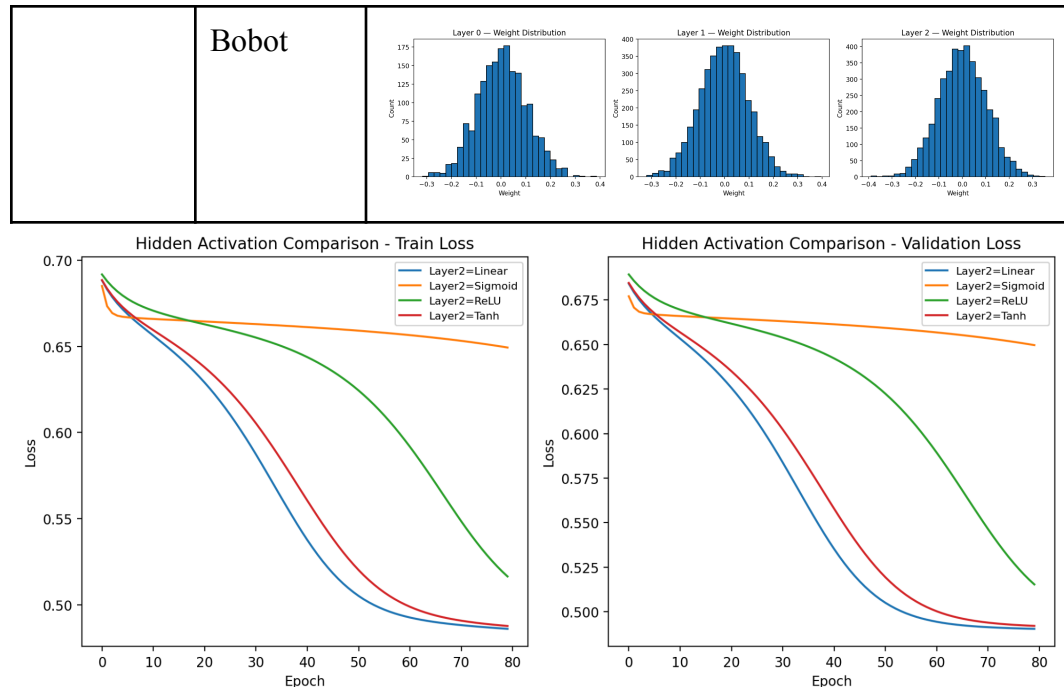
2.2.2 Pengaruh Fungsi Aktivasi

Dilakukan pengujian untuk seluruh variasi fungsi aktivasi (kecuali *softmax*), dengan konfigurasi 3 layer dengan width 64, dan dipilih *hidden layer kedua* sebagai *test layer*. Hidden layer yang lain menggunakan fungsi aktivasi ReLU.

Activation Function	Accuracy	Precision	Recall	F1
Tanh	0.748	0.7764	0.8294	0.8020
ReLU	0.7475	0.7545	0.8740	0.8099
Linear	0.747	0.7756	0.8285	0.8012
Sigmoid	0.6155	0.6155	1.0	0.7619

Activation Function	Distribusi	Gambar
---------------------	------------	--------

Tanh	Gradien	
	Bobot	
ReLU	Gradien	
	Bobot	
Linear	Gradien	
	Bobot	
Sigmoid	Gradien	

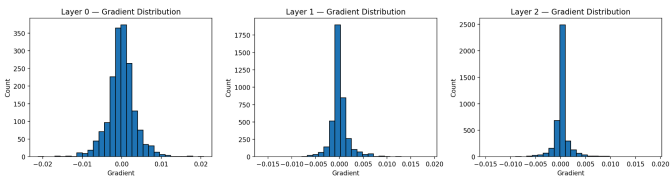
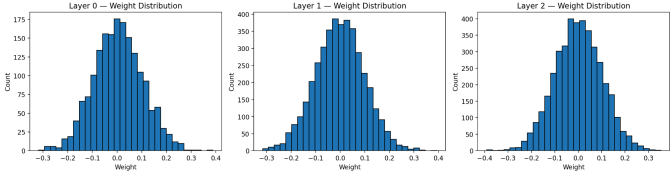
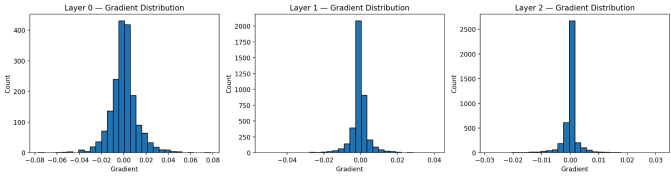
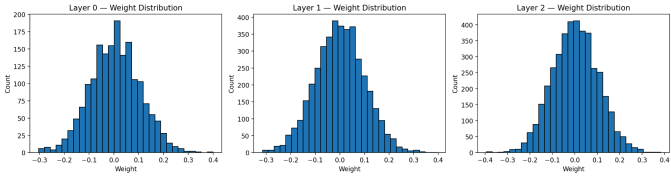
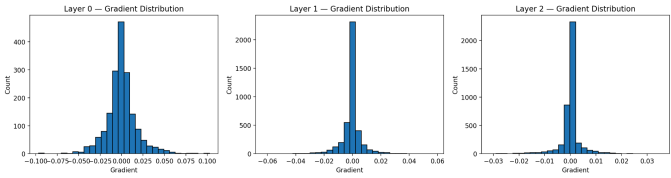
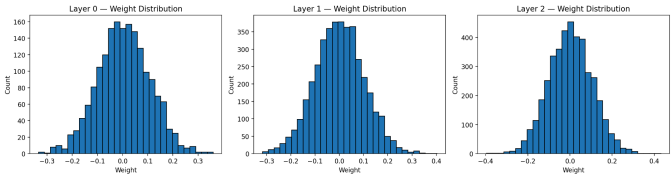


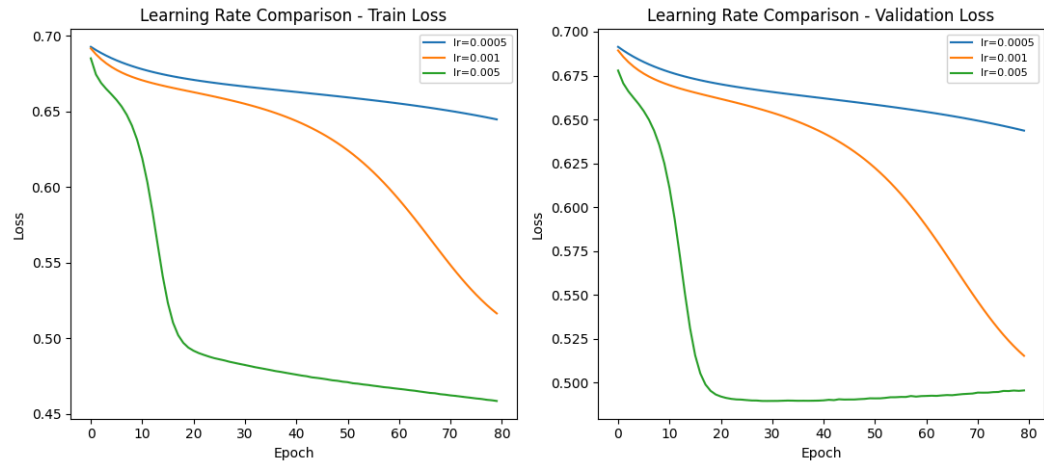
Perubahan fungsi aktivasi pada satu hidden layer tidak terlalu memberikan dampak pada performa model (terkecuali sigmoid). Hal ini menunjukkan perubahan pada satu layer saja tidak terlalu berpengaruh pada performa model. Namun, untuk sigmoid, karena sigmoid mengubah nilai menjadi diantara 0 dan 1, maka gradiennya menjadi sangat kecil, membuat perubahan bobot menjadi sangat kecil pula bahkan stagnan. Jadi, dengan epoch yang terbatas, model ini belum banyak belajar sehingga performanya masih kurang baik.

2.2.3 Pengaruh Learning Rate

Dilakukan pengujian untuk tiga variasi learning rate yaitu 0.0005, 0.001, dan 0.005.

Learning Rate	Accuracy	Precision	Recall	F1
0.0005	0.0005	0.6155	1.0	0.7619931909625502
0.001	0.7475	0.7545582047685835	0.8740861088545898	0.8099360180654874
0.005	0.738	0.772972972972973	0.8131600324939074	0.7925574030087095

Learning Rate	Distribusi	Gambar
0.0005	Gradien	
	Bobot	
0.001	Gradien	
	Bobot	
0.005	Gradien	
	Bobot	



Learning rate yang besar (0.005) sangat cepat konvergen, terlihat dari grafik *loss* nya, tetapi setelah 20 epoch melandai, menandakan model sudah tidak belajar lagi. Selain itu, *learning rate* yang kecil (0.0005) menyebabkan proses model belajar sangat lambat (sangat melandai pada grafik *loss*) sehingga dengan epoch yang kecil, model juga belum belajar banyak.

Learning rate yang menengah (0.001) justru memberikan performa model yang paling baik diantara *learning rate* yang lain. Dilihat dari grafik *loss*, kecepatan model belajar relatif stabil, tidak langsung curam di awal, tapi masih mengalami penurunan signifikan sepanjang pembelajaran sehingga proses pembelajaran terus berlangsung sepanjang 80 epoch.

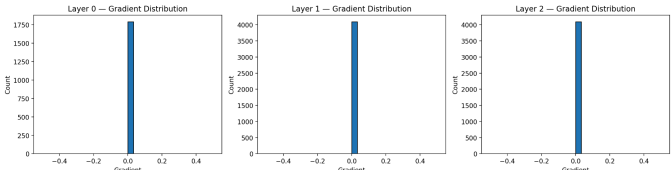
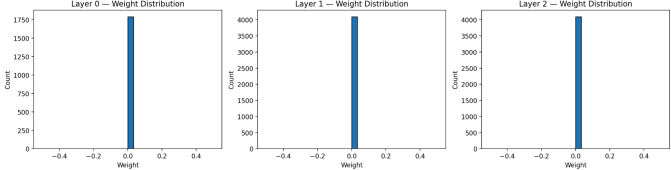
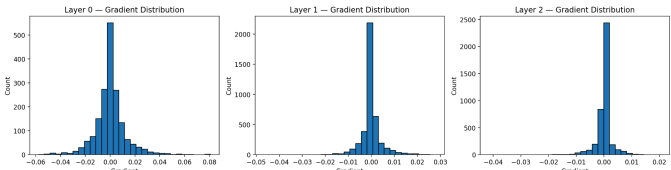
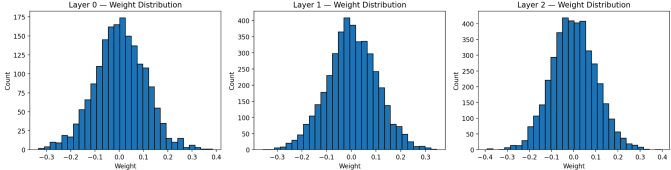
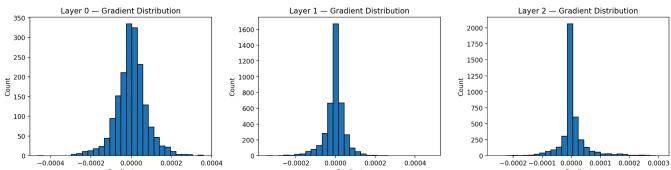
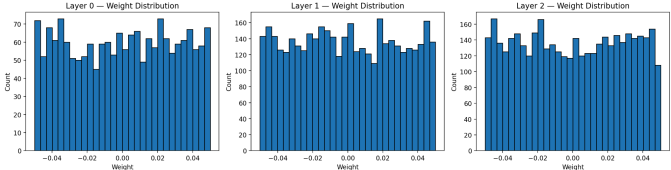
2.2.4 Pengaruh Inisialisasi Bobot

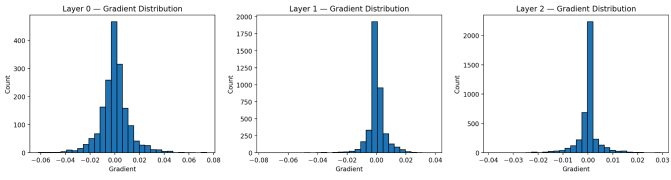
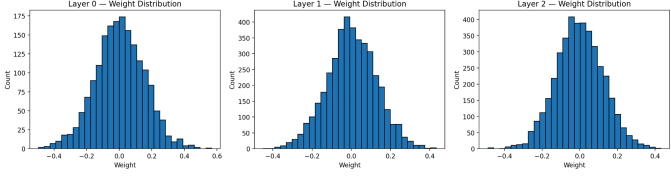
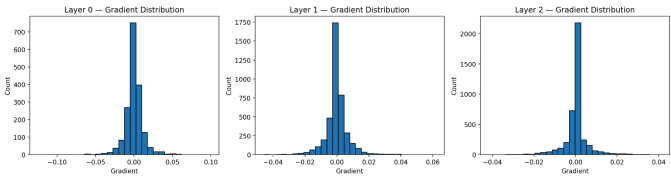
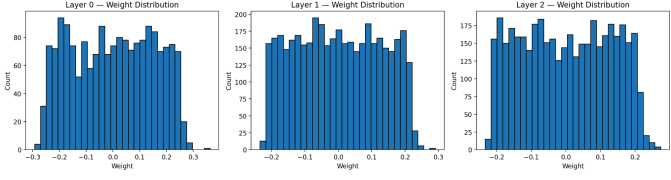
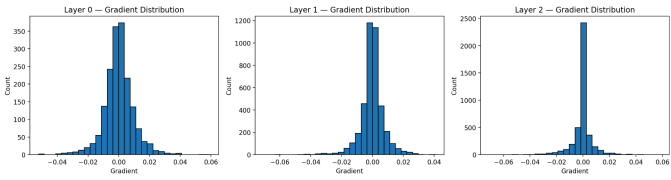
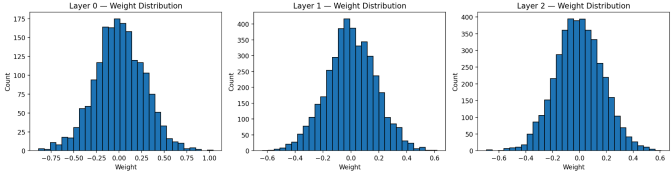
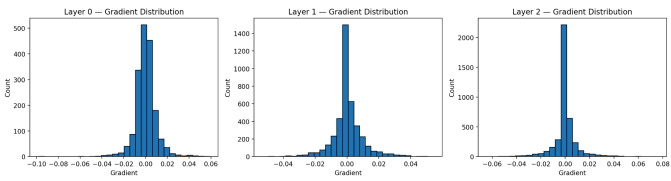
Dilakukan pengujian untuk setiap cara inisialisasi bobot, dengan nilai parameter sebagai berikut.

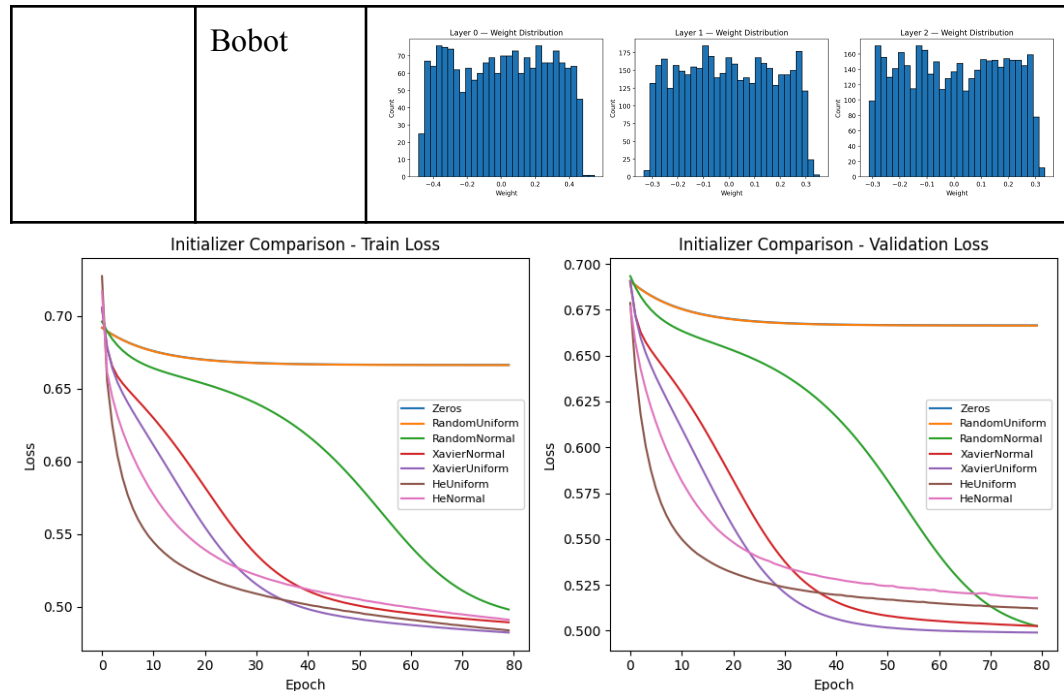
- mean = 0.0
- variance = 0.01
- low = -0.5
- high = 0.5

Initializer	Accuracy	Precision	Recall	F1
Zero	0.6155	0.6155	1.0	0.7619
Random (Normal)	0.7475	0.7729	0.8350	0.8028
Random (Uniform)	0.6155	0.6155	1.0	0.7619
Xavier (Normal)	0.7435	0.7707	0.8302	0.7993

Xavier (Uniform)	0.748	0.7739	0.8342	0.8029
He (Normal)	0.732	0.7606	0.8237	0.7909
He (Uniform)	0.738	0.7651	0.8285	0.7956

Initializer	Distribusi	Gambar
Zero	Gradien	
	Bobot	
Random (Normal)	Gradien	
	Bobot	
Random (Uniform)	Gradien	
	Bobot	

Xavier (Normal)	Gradien	
	Bobot	
Xavier (Uniform)	Gradien	
	Bobot	
He (Normal)	Gradien	
	Bobot	
He (Uniform)	Gradien	



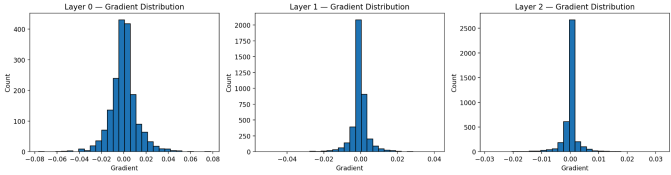
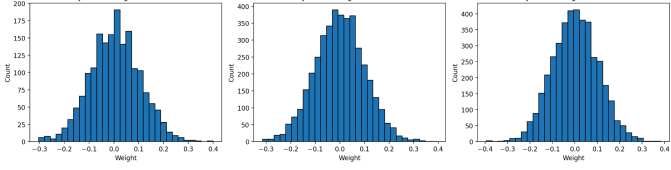
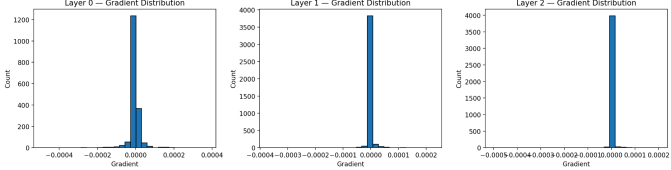
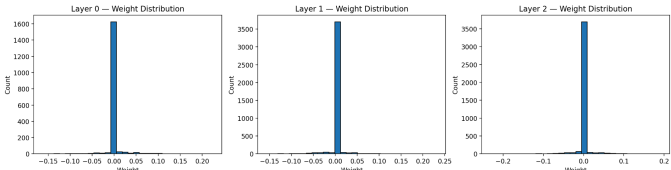
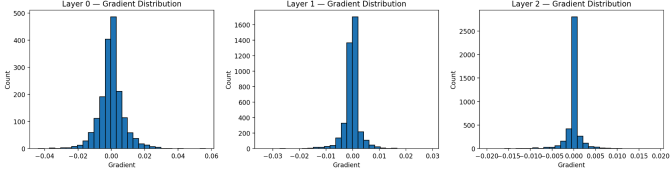
Berdasarkan nilai F1 dan akurasi model, inisialisasi bobot *zero initialization* dan random dengan distribusi uniform memiliki hasil yang terburuk. Hasil ini seperti demikian karena pada *zero initialization*, semua lapisan neuron memiliki bobot yang sama sehingga gradien menjadi terlalu mirip. Sedangkan itu, inisialisasi random dengan distribusi uniform secara efektif menghilangkan menjadikan bobot kurang dari 0 menjadi 0 karena menggunakan fungsi aktivasi ReLU sehingga masalah yang serupa terjadi pada metode inisialisasi bobot ini.

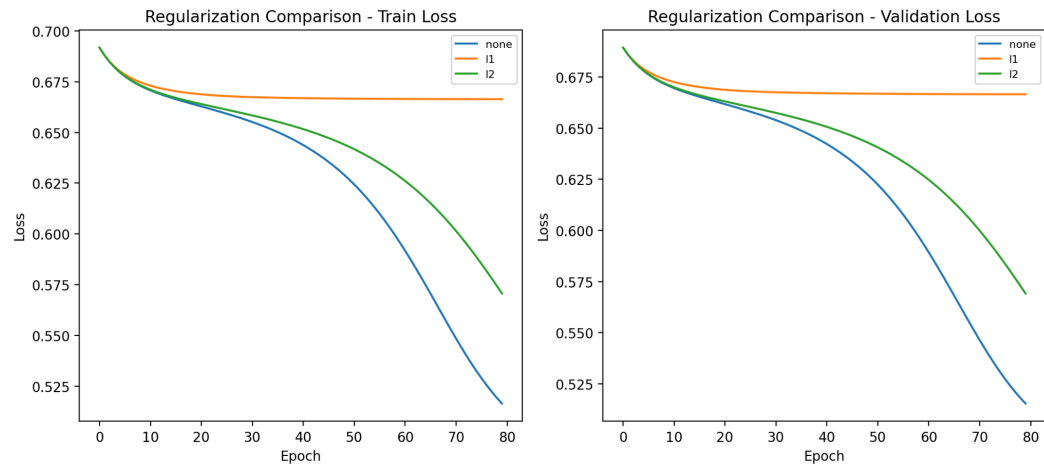
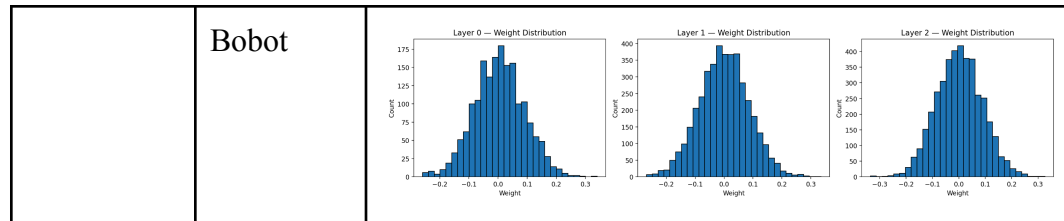
Di sisi lainnya, metode-metode inisialisasi bobot lainnya memiliki hasil kinerja yang cukup sebanding satu dengan lainnya dengan performa yang lebih baik dibandingkan dua inisialisasi bobot sebelumnya. Hal ini karena metode-metode inisialisasi bobot lainnya tersebut mampu mengacak nilai bobot dengan skala yang lebih besar sehingga memberikan nilai-nilai yang lebih bervariasi untuk fungsi aktivasi ReLU. Secara teori, metode-metode inisialisasi bobot He memiliki keunggulan dengan ReLU karena variansinya tetap konsisten meski terpotong setengahnya. Namun, *learning rate* yang relatif kecil dan jumlah epoch terbatas mengakibatkan metode inisialisasi Xavier mampu mendekati konvergen lebih cepat dibandingkan metode inisialisasi He seperti yang dapat dilihat pada kurva loss validasi.

2.2.5 Pengaruh Regularisasi

Dilakukan pengujian untuk tiga variasi regularisasi yaitu tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.

Regularization	Accuracy	Precision	Recall	F1
none	0.7475	0.7545	0.8740	0.8099
l1	0.6155	0.6155	1.0	0.7619
l2	0.7215	0.7013	0.9536	0.8082

Learning Rate	Distribusi	Gambar
none	Gradien	
	Bobot	
l1	Gradien	
	Bobot	
l2	Gradien	

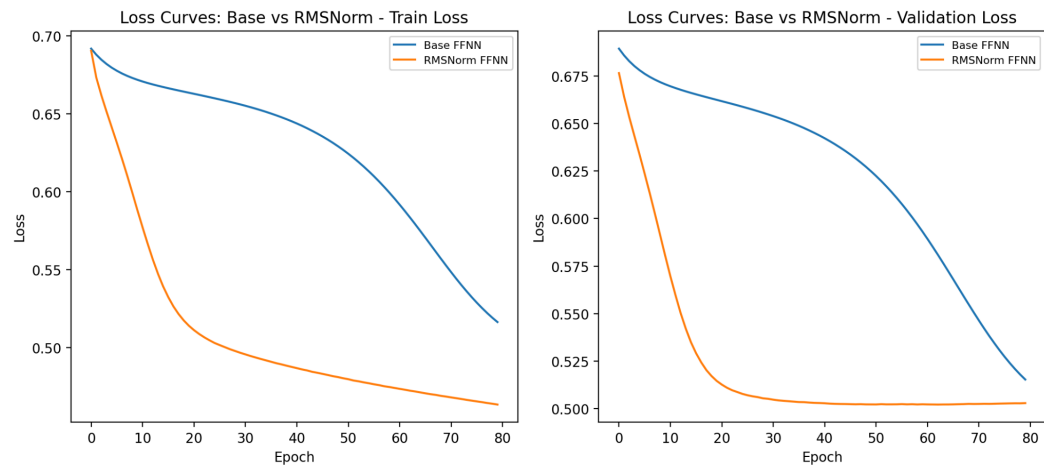
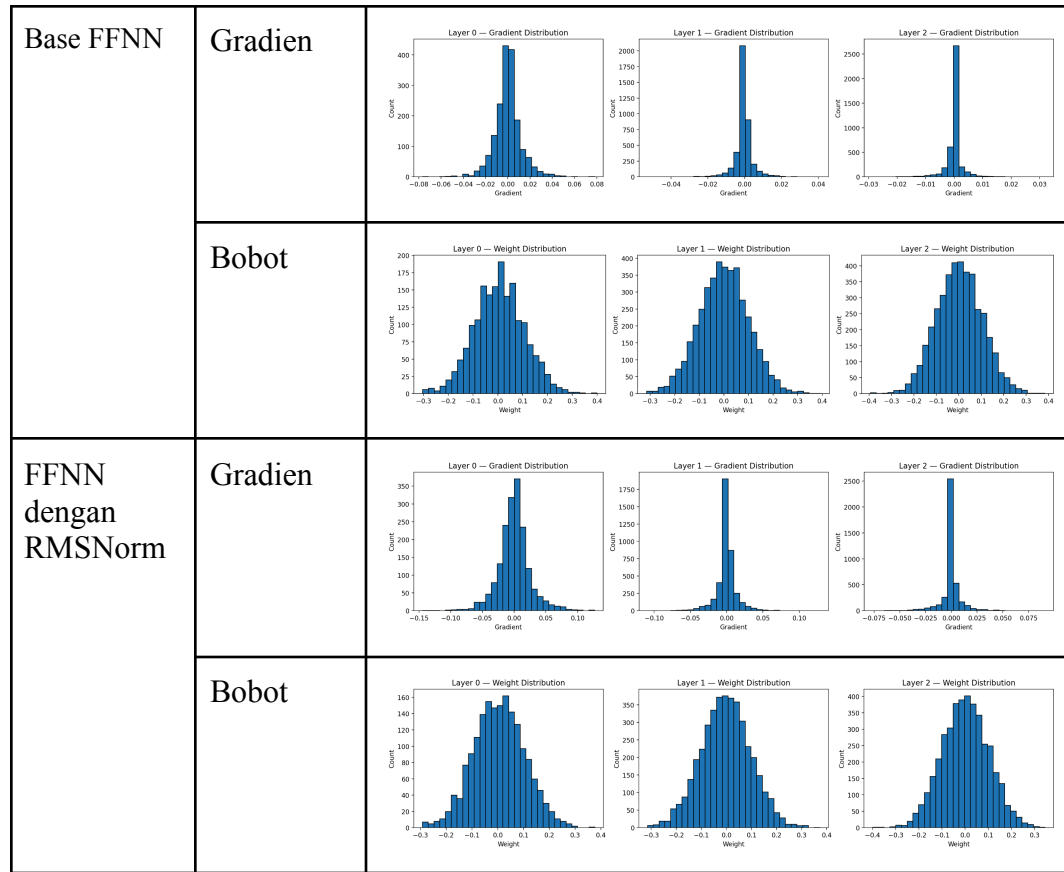


Konfigurasi tanpa regularisasi memberikan performa terbaik, diikuti oleh L2, dan kemudian L1. Jika dilihat dari distribusi bobotnya, L1 menghasilkan sparsity yang sangat kuat karena banyak bobot terdorong hingga ke angka nol. Dari sini, dapat diinferensikan bahwa L1 “menghapus” terlalu banyak fitur sehingga menurunkan kapasitas efektif model secara drastis sehingga performanya menurun. Di sisi lain, L2 menekan besaran bobot, namun tidak seagresif L1 dalam mematikan koneksi antar-neuron. Hal ini membuat L2 masih lebih unggul dibanding L1, meski hasilnya belum bisa melampaui model tanpa regularisasi.

2.2.6 Pengaruh Normalisasi RMSNorm

Model	Accuracy	Precision	Recall	F1
Base FFNN	0.7475	0.7545	0.8740	0.8099
FFNN dengan RMSNorm	0.7455	0.7751	0.8261	0.7998

Model	Distribusi	Gambar
-------	------------	--------



Berdasarkan hasil uji coba model FFNN dengan RMSNorm, didapatkan bahwa model tersebut memiliki performa yang tidak jauh berbeda dengan model FFNN tanpa RMSNorm. Bahkan, dari segi skor F1, model FFNN dengan RMSNorm justru memiliki performa yang sedikit lebih buruk dibandingkan yang tidak. Padahal, kurva loss menunjukkan proses pelatihan model dengan RMSNorm dapat stabil lebih cepat. Hal ini dapat terjadi karena RMSNorm melakukan normalisasi pada *net input* lapisan supaya menghindari nilai-nilai ekstrem yang dapat berdampak pada gradien menjadi tidak stabil. Namun,

parameter dan konfigurasi model dengan inisialisasi bobot yang relatif stabil dan *learning rate* yang relatif rendah menyebabkan fungsionalitas RMSNorm tidak dapat bekerja dengan optimal. Justru, normalisasi RMSNorm membuat nilai-nilai *input* kehilangan “karakteristiknya” sehingga mengurangi kinerja model. Alhasil, model tanpa RMSNorm dapat memiliki performa yang relatif sebanding dengan model yang menggunakan RMSNorm.

2.2.7 Perbandingan dengan *Library* sklearn

Model	Accuracy	Precision	Recall	F1
FFNN <i>from scratch</i>	0.7475	0.7545	0.8740	0.8099
MLPClassifier	0.725	0.7739	0.7814	0.7776

Berdasarkan hasil yang diperoleh, model FFNN *from scratch*, terutama dalam hal recall dan F1-score. Hal ini menunjukkan bahwa model lebih mampu mendeteksi kelas positif dengan lebih sedikit false negatif. Namun, precision yang sedikit lebih rendah mengindikasikan adanya peningkatan false positive.

Meskipun kedua model menggunakan konfigurasi arsitektur dan hyperparameter yang serupa, terdapat perbedaan implementasi internal yang menyebabkan perbedaan performa. Model MLPClassifier dari sklearn menggunakan optimasi SGD dengan mekanisme tambahan seperti learning rate scheduling dan kemungkinan *early stopping*, serta inisialisasi bobot yang lebih terstandarisasi. Sementara itu, model FFNN yang diimplementasikan secara manual cenderung menggunakan prosedur *update* gradien yang lebih sederhana tanpa mekanisme tambahan tersebut.

III. KESIMPULAN DAN SARAN

3.1. Kesimpulan

Model *Feedforward Neural Network* (FFNN) berhasil diimplementasikan dengan baik dan memberikan performa yang baik dalam memprediksi dataset *Global Student Placement & Salary Dataset* dengan rata-rata akurasi prediksi sekitar 74 persen. Hasil eksperimen menunjukkan bahwa variasi *hyperparameter* seperti jumlah neuron (width) *learning rate*, dan regularisasi sangat mempengaruhi kemampuan model. Namun, parameter lainnya seperti variasi fungsi aktivasi pada *hidden layer* tertentu atau metode inisialisasi bobot spesifik tidak memberikan pengaruh yang terlalu signifikan terhadap hasil akhir akurasi prediksi pada pengujian ini. Secara keseluruhan, model FFNN yang diimplementasikan mampu memberikan hasil yang kompetitif dengan model *MLPClassifier* dari pustaka *scikit-learn*.

3.2. Saran

1. Melakukan pengujian dengan lebih banyak uji kasus untuk memperkuat hasil analisa.
2. Melakukan pengujian terhadap fungsi *loss* untuk mengetahui pengaruh fungsi *loss* terhadap model.

IV. PEMBAGIAN TUGAS

NIM	Nama	Peran
13523089	Ahmad Ibrahim	Pembuatan bonus <i>automatic differentiation</i> , integrasi model, pembuatan laporan.
13523102	Michael Alexander Angkawijaya	Pengujian <i>hyperparameter</i> , pembuatan kerangka model, pembuatan laporan.
13523112	Aria Judhistira	Pembuatan fungsi aktivasi dan loss, integrasi model, pembuatan bonus inisialisasi bobot dan RMSNorm, pembuatan laporan.

V. REFERENSI

- ApX Machine Learning. (n.d.). *Kaiming (He) Initialization*. ApX Machine Learning. Retrieved March 15, 2026, from <https://apxml.com/courses/how-to-build-a-large-language-model/chapter-12-initialization-techniques-deep-networks/kaiming-he-initialization>
- GeeksforGeeks. (2025, July 23). *Xavier initialization*. GeeksforGeeks. Retrieved March 18, 2026, from <https://www.geeksforgeeks.org/deep-learning/xavier-initialization/>
- Osajima, J. (2018, July 18). *The Math behind Neural Networks - Backpropagation*. Retrieved March 14, 2026, from <https://www.jasonosajima.com/backprop>
- Osajima, J. (2018, July 18). *The Math behind Neural Networks - Forward Propagation*. Retrieved March 14, 2026, from <https://www.jasonosajima.com/forwardprop>
- PyTorch. (n.d.). *RMSNorm — PyTorch 2.10 documentation*. PyTorch documentation. Retrieved March 18, 2026, from <https://docs.pytorch.org/docs/stable/generated/torch.nn.modules.normalization.RMSNorm.html>

VI. LAMPIRAN

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Ahmad Ibrahim / 13523089

Michael Alexander Angkawijaya / 13523102

Aria Judhistira / 13523112

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Laporan Tugas Besar 1 IF3170 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

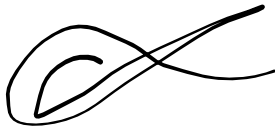
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1

Lainnya, Jelaskan:		
--------------------	--	--

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026



Ahmad Ibrahim

13523089



Michael Alexander Angkawijaya

13523102



Aria Judhistira

13523112